DESENVOLVIMENTO DE SOFTWARE VISUAL · 2026

Lista de Exercícios

C# · Orientação a Objetos · ASP.NET Core MVC · EF Core · API

72 exercícios em 10 níveis, do zero até um pouco além do que já vimos em aula. Cada exercício traz o enunciado, o que ele treina e um critério objetivo de "está pronto quando...".

Como usar esta lista

- **Faça na ordem.** Os níveis são cumulativos: o 6 assume que você fez o 2 e o 3.
- **Digite o código, não copie.** O objetivo é errar, ler a mensagem de erro e corrigir — é assim que se aprende.
- **Um projeto por nível.** Níveis 1 a 4 são projetos de console (`dotnet new console`). Do 5 em diante, um projeto web (`dotnet new mvc`).
- **Commite tudo.** Crie um repositório `exercicios-csharp` e faça um commit por exercício. Serve de treino de Git e de portfólio.
- **Não precisa terminar tudo antes da próxima aula.** Meta razoável: níveis 1–4 na primeira semana, 5–6 na segunda, 7–8 na terceira.

Marcação	Significado
●○○	Fundamento. Se travar aqui, revise a aula antes de seguir.
●●○	Aplicação. Junta duas ou três coisas que você já viu.
●●●	Integração. Exige pensar na estrutura, não só na sintaxe.
Desafio	Opcional. Vai além do conteúdo da disciplina — faça se sobrar tempo.

Índice dos níveis

Nível 0 Preparação do ambiente e Git · 3 exercícios

Nível 1 Fundamentos de C#: tipos, condições, laços, métodos · 9 exercícios

Nível 2 POO parte 1: classes, objetos, construtores, encapsulamento · 8 exercícios

Nível 3 POO parte 2: herança, interfaces, polimorfismo, camadas · 8 exercícios

Nível 4 Coleções e LINQ · 8 exercícios

Nível 5 Primeiro projeto web: MVC, Controllers e Views · 8 exercícios

Nível 6 Banco de dados: EF Core + SQLite e migrations · 8 exercícios

Nível 7 API REST: CRUD completo em JSON · 8 exercícios

Nível 8 Frontend consumindo a API com `fetch` · 7 exercícios

Nível 9 Um pouco além: serviços, API externa, paginação · 5 exercícios

Final Projeto integrador: sistema de biblioteca

Nível 0 — Preparação

Antes de programar: garantir que a máquina está pronta e que você sabe versionar.

0.1 Ambiente instalado

●○○

Instale o **.NET SDK 8**, o **Git** e o **VS Code** (com as extensões **C#** e **C# Dev Kit**). Depois, abra um terminal e rode os três comandos de verificação.

```
dotnet --version
git --version
dotnet --list-sdks
```

Está pronto quando: `dotnet --version` mostra `8.x` e os outros dois comandos não dão erro.

0.2 Primeiro programa

●○○

Crie um projeto de console chamado `Exercicios`, abra no VS Code e rode. Depois altere a mensagem para o seu nome e rode de novo.

```
dotnet new console -n Exercicios
cd Exercicios
code .
dotnet run
```

Está pronto quando: o terminal imprime a sua mensagem personalizada.

0.3 Repositório de exercícios

●○○

Crie um repositório no GitHub chamado `exercicios-csharp`, conecte a pasta local e faça o primeiro push. Crie também um branch chamado `nivel-1` e volte para a `main`.

```
git init
git add .
git commit -m "Primeiro commit"
git branch -M main
git remote add origin <url-do-seu-repo>
git push -u origin main
```

Está pronto quando: os arquivos aparecem no GitHub e `git branch` lista `main` e `nivel-1`.

Nível 1 — Fundamentos de C#

Tipos, entrada e saída, condições, laços de repetição e métodos. Tudo em projeto de console.

1.1 Conversor de temperatura ●○○

Peça ao usuário uma temperatura em Celsius e mostre o valor em Fahrenheit e em Kelvin. Fórmulas: `F = C × 9/5 + 32` e `K = C + 273.15`.

Treina: `Console.ReadLine()`, conversão de texto para número (`double.Parse`), interpolação de string (`$"..."`).

Está pronto quando: digitando `25` a saída mostra `77` °F e `298,15` K.

1.2 Classificador de faixa etária ●○○

Leia uma idade e classifique: **0–12** criança, **13–17** adolescente, **18–59** adulto, **60+** idoso. Se a idade for negativa, mostre uma mensagem de erro.

Treina: `if / else if / else`, operadores de comparação e lógicos.

Está pronto quando: os cinco casos (incluindo o negativo) dão a resposta certa. Teste os limites: 12, 13, 17, 18, 59, 60.

1.3 Calculadora com `switch` ●○○

Leia dois números e um operador (`+`, `-`, `*`, `/`) e mostre o resultado. Use `switch`. Trate a divisão por zero.

Treina: `switch`, tratamento de caso inválido (`default`).

Está pronto quando: as quatro operações funcionam, dividir por zero mostra aviso em vez de quebrar, e um operador inválido cai no `default`.

1.4 Tabuada ●○○

Peça um número e imprima a tabuada dele de 1 a 10. Depois, faça uma segunda versão que imprima **todas** as tabuadas de 1 a 10 usando laços aninhados.

Treina: `for`, laço dentro de laço, formatação de saída.

Está pronto quando: a segunda versão imprime 100 linhas organizadas, agrupadas por tabuada.

1.5 Adivinhe o número ●●○

O programa sorteia um número de 1 a 100 (`new Random().Next(1, 101)`) e o usuário tenta adivinhar. A cada tentativa, responda "maior" ou "menor". Ao acertar, mostre em quantas tentativas conseguiu.

Treina: `while`, contador, `Random`.

Está pronto quando: o laço só termina no acerto e o número de tentativas está correto.

1.6 Boletim escolar ●●○

Leia 4 notas, guarde em um `double[]` e calcule: média, maior nota, menor nota e a situação (média ≥ 7 aprovado; entre 5 e 7 recuperação; abaixo de 5 reprovado). **Cada cálculo deve estar em um método separado.**

Treina: arrays, `foreach`, métodos com parâmetro e retorno.

Está pronto quando: o `Main` praticamente só lê os dados e chama métodos — nenhum cálculo solto dentro dele.

1.7 Análise de texto ●●○

Leia uma frase e mostre: quantidade de caracteres, de palavras, de vogais, a frase em maiúsculas e a frase invertida.

Treina: `string` (`Length`, `Split`, `ToUpper`), `foreach` em `char`, `char.ToLower`.

Está pronto quando: "Ola Mundo" retorna 9 caracteres, 2 palavras e 4 vogais.

1.8 Validador de senha ●●○

Leia uma senha e valide as regras: mínimo 8 caracteres, pelo menos uma maiúscula, uma minúscula, um número e um caractere especial. Mostre **quais** regras falharam, não apenas "senha inválida".

Treina: métodos que retornam `bool`, decomposição de um problema em partes pequenas.

Está pronto quando: existe um método `bool` por regra e a senha `abc` lista 4 falhas.

1.9 Menu de console reutilizável Desafio

Junte os exercícios 1.1 a 1.8 em um único programa com um menu numerado. O programa só encerra quando o usuário escolher "0 - Sair", voltando ao menu depois de cada operação.

Treina: `while` com condição de saída, `switch` chamando métodos, organização do código.

Está pronto quando: dá para rodar qualquer exercício várias vezes seguidas sem reiniciar o programa.

Nível 2 — POO parte 1

Classes como moldes, objetos como instâncias. Construtores, propriedades e encapsulamento.

2.1 Sua primeira classe

●○○

Crie a classe `Produto` com as propriedades `Nome`, `Preco` e `Quantidade`. No `Main`, crie três produtos e imprima os dados de cada um.

Treina: `class`, propriedades com `get; set;`, `new`.

Está pronto quando: a classe está em um arquivo próprio (`Produto.cs`) e os três objetos imprimem valores diferentes.

2.2 Construtor

●○○

Adicione à `Produto` um construtor que receba nome, preço e quantidade. Crie os objetos passando os valores direto no `new`.

```
var p = new Produto("Teclado", 150.00m, 3);
```

Treina: construtor, atribuição de propriedades.

Está pronto quando: não é mais possível criar um produto sem informar os três dados.

2.3 Métodos e propriedade calculada

●○○

Na `Produto`, adicione o método `decimal ValorTotal()` (preço × quantidade) e a propriedade somente-leitura `bool EmEstoque` (verdadeira se a quantidade for maior que zero).

```
public bool EmEstoque => Quantidade > 0;
```

Treina: métodos de instância, propriedade calculada.

Está pronto quando: um produto com quantidade 0 retorna `EmEstoque == false` e `ValorTotal() == 0`.

2.4 Conta bancária (encapsulamento)

●●○

Crie a classe `ContaBancaria` com `Titular` e `Saldo`. O saldo **só pode ser lido de fora** — nunca alterado diretamente. Adicione os métodos `Depositar(valor)` e `Sacar(valor)`, que validam: valor tem que ser positivo, e o saque não pode deixar o saldo negativo.

```
public decimal Saldo { get; private set; }
```

Treina: encapsulamento, `private set`, validação dentro da classe.

Está pronto quando: a linha `conta.Saldo = 1000000;` **não compila**, e sacar mais do que tem retorna erro sem alterar o saldo.

2.5 `ToString()` ●●○

Sobrescreva o método `ToString()` em `Produto` para devolver algo legível, como `"Teclado — R$ 150,00 (3 un.)"`. Depois teste imprimindo o objeto direto: `Console.WriteLine(produto)`.

Treina: `override`, o conceito de que toda classe herda de `object`.

Está pronto quando: imprimir o objeto direto mostra o texto formatado em vez de `Exercicios.Produto`.

2.6 Enum + `switch` ●●○

Crie o enum `StatusPedido` com os valores `Aguardando`, `Pago`, `Enviado`, `Entregue` e `Cancelado`. Crie a classe `Pedido` com esse status e um método `string DescreverStatus()` que use `switch` para devolver uma frase amigável.

Treina: `enum`, `switch` sobre enum.

Está pronto quando: os cinco status retornam frases diferentes.

2.7 Sobrecarga de métodos ●●○

Na `ContaBancaria`, crie duas versões de `Transferir`: uma que recebe `(ContaBancaria destino, decimal valor)` e outra que recebe também uma `string descricao`. A primeira deve chamar a segunda passando uma descrição padrão.

Treina: sobrecarga (mesmo nome, assinaturas diferentes), reaproveitamento de código.

Está pronto quando: as duas chamadas funcionam e a lógica de transferência existe em **um só lugar**.

2.8 Cadastro em memória ●●●

Monte um programa de console com menu que permita: **cadastrar** produto, **listar** todos, **buscar** por nome, **remover** por nome e mostrar o **valor total** do estoque. Guarde tudo em uma `List<Produto>`.

Treina: `List<T>`, `Add`, `Remove`, `foreach`, integração de tudo do nível.

Está pronto quando: dá para cadastrar 3 produtos, remover 1, listar e ver o total correto — tudo sem reiniciar o programa.

Nível 3 — POO parte 2

Herança, classes abstratas, interfaces, polimorfismo e separação em camadas. **Isto é o conteúdo da aula de 15/09 — você está se adiantando.**

3.1 Herança ●○○

Crie a classe `Animal` com `Nome`, `Idade` e o método `Apresentar()`. Crie `Cachorro` e `Gato` herdando de `Animal`, cada uma com uma propriedade própria (ex.: `Raca` e `PelagemLonga`).

```
public class Cachorro : Animal { ... }
```

Treina: `:` (herança), reaproveitamento de membros da classe base.

Está pronto quando: um `Cachorro` consegue usar `Apresentar()` sem que ele tenha sido escrito na classe `Cachorro`.

3.2 `virtual` e `override` ●●○

Adicione à `Animal` o método `virtual string EmitirSom()` devolvendo "...". Sobrescreva em `Cachorro` ("Au au!") e em `Gato` ("Miau!"). Depois, sobrescreva `Apresentar()` em `Cachorro` chamando `base.Apresentar()` e acrescentando a raça.

Treina: `virtual`, `override`, `base`.

Está pronto quando: cada animal emite seu som e o `Apresentar()` do cachorro mostra a informação da classe base **mais** a raça.

3.3 Classe abstrata ●●○

Crie a classe abstrata `FormaGeometrica` com `abstract double CalcularArea()` e `abstract double CalcularPerimetro()`. Implemente `Circulo`, `Retangulo` e `Triangulo`.

Treina: `abstract` — a classe define o contrato mas não a implementação.

Está pronto quando: a linha `new FormaGeometrica()` **não compila** (é abstrata), mas as três filhas funcionam.

3.4 Polimorfismo ●●○

Crie uma `List<FormaGeometrica>` com objetos dos três tipos. Percorra a lista com um único `foreach` imprimindo a área de cada uma — **sem usar `if` para descobrir o tipo**.

Treina: polimorfismo — o mesmo código funciona para tipos diferentes.

Está pronto quando: você adiciona uma `Losango` nova e ela funciona no `foreach` sem alterar **nenhuma** linha do laço.

3.5 Interface

●●○

Crie a interface `IPagamento` com o método `bool Processar(decimal valor)`. Implemente `PagamentoCartao` (recusa valores acima de 5000), `PagamentoPix` (aprova sempre) e `PagamentoBoleto` (recusa valores abaixo de 10). Crie um método que receba `IPagamento` e processe.

Treina: `interface`, programar contra abstração e não contra implementação.

Está pronto quando: o método que processa o pagamento **não sabe** qual é a forma concreta — só conhece a interface.

3.6 Organização em pastas e namespaces

●●○

Reorganize os exercícios do nível 3 em pastas: `Models/` (as classes de domínio), `Services/` (as classes com regras) e `Program.cs` na raiz. Ajuste os `namespace` para bater com as pastas e adicione os `using` necessários.

Treina: `namespace`, organização de projeto — exatamente o que fazemos no PokéDraft.

Está pronto quando: o projeto compila e **nenhuma** classe está solta na raiz além do `Program.cs`.

3.7 Repositório genérico em memória

●●●

Crie a interface `IRepositorio<T>` com `Adicionar(T)`, `ListarTodos()`, `BuscarPorId(int)` e `Remover(int)`. Implemente `RepositorioMemoria<T>` usando uma `List<T>` interna.

Treina: genéricos (`<T>`), interface com implementação trocável. É o mesmo papel que o `DbContext` cumpre no projeto real.

Está pronto quando: o mesmo repositório funciona para `Produto` e para `Animal` sem duplicar código.

3.8 Injeção de dependência manual

●●●

Crie a classe `ProdutoService` que recebe um `IRepositorio<Produto>` **pelo construtor** e expõe métodos com regras (ex.: `Cadastrar` que rejeita preço negativo). No `Main`, monte o serviço passando o repositório.

```
var repo = new RepositorioMemoria<Produto>();
var service = new ProdutoService(repo);
```

Treina: injeção de dependência — o padrão que o ASP.NET Core faz automaticamente nos Controllers.

Está pronto quando: o `ProdutoService` não tem nenhum `new` de repositório dentro dele.

Nível 4 — Coleções e LINQ

Manipular listas de objetos. É o que você vai usar em **toda** consulta ao banco depois.

Para todos os exercícios deste nível, use uma lista com pelo menos 10 produtos de categorias variadas, criada direto no código.

4.1 Operações básicas de lista

●○○

Com a `List<Produto>`: adicione um item, remova outro, conte quantos restaram, verifique se um produto específico existe e limpe a lista ao final.

Treina: `Add`, `Remove`, `Count`, `Contains`, `Clear`.

Está pronto quando: você imprime a contagem depois de cada operação e os números batem.

4.2 Filtrar e ordenar

●●○

Usando LINQ, monte três consultas: produtos com preço acima de 100; produtos ordenados por preço decrescente; produtos de uma categoria específica ordenados por nome.

```
var caros = produtos.Where(p => p.Preco > 100)
                    .OrderByDescending(p => p.Preco)
                    .ToList();
```

Treina: `Where`, `OrderBy`, `OrderByDescending`, expressões lambda.

Está pronto quando: as três consultas cabem em uma linha cada e retornam o esperado.

4.3 Projeção com `Select`

●●○

Monte uma consulta que devolva apenas **o nome e o valor total** de cada produto (não o objeto inteiro), usando um tipo anônimo. Imprima o resultado.

```
var resumo = produtos.Select(p => new { p.Nome, Total = p.Preco * p.Quantidade });
```

Treina: `Select`, tipo anônimo. É a base do conceito de **DTO** que você usa no nível 7.

Está pronto quando: a saída mostra só as duas informações, sem os demais campos.

4.4 Agregações

●●○

Calcule com LINQ: valor total do estoque (`Sum`), preço médio (`Average`), produto mais caro (`Max` ou `OrderByDescending().First()`), quantos estão sem estoque (`Count`) e se existe algum produto acima de R$ 1000 (`Any`).

Treina: `Sum`, `Average`, `Max`, `Min`, `Count`, `Any`, `All`.

Está pronto quando: nenhum desses cálculos usa `foreach`.

4.5 Busca segura

●●○

Busque um produto por nome usando `FirstOrDefault`. Trate o caso em que ele não existe, imprimindo "Produto não encontrado" em vez de quebrar o programa.

Treina: `FirstOrDefault` vs `First`, checagem de `null`.

Está pronto quando: buscar um nome inexistente mostra a mensagem e o programa continua rodando.

4.6 Agrupamento

●●●

Agrupe os produtos por categoria e, para cada grupo, imprima: nome da categoria, quantidade de produtos e valor total daquela categoria.

```
var porCategoria = produtos.GroupBy(p => p.Categoria);
```

Treina: `GroupBy`, iterar sobre grupos.

Está pronto quando: a soma das quantidades de todos os grupos é igual ao total de produtos.

4.7 Dicionário

●●○

Leia uma frase e conte quantas vezes cada palavra aparece, usando `Dictionary<string, int>`. Depois imprima ordenado da mais frequente para a menos.

Treina: `Dictionary`, `ContainsKey`, chave/valor.

Está pronto quando: a frase "o rato roeu o rato" mostra `o = 2` e `rato = 2`.

4.8 Relatório completo

●●●

Monte um único método `GerarRelatorio(List<Produto>)` que imprima um relatório formatado com: total de itens, valor do estoque, top 3 mais caros, produtos sem estoque e a distribuição por categoria.

Treina: encadear várias consultas, `Take`, formatação de saída.

Está pronto quando: o relatório sai organizado e legível, com todas as cinco seções.

Nível 5 — Primeiro projeto web (MVC)

A partir daqui, projeto web. Crie um novo: `dotnet new mvc -n LojaWeb`.

5.1 Criar e rodar

●○○

Crie o projeto `LojaWeb`, rode e abra no navegador. Depois altere o texto da página inicial e o título da aba.

```
dotnet new mvc -n LojaWeb
cd LojaWeb
dotnet run
```

Treina: estrutura do projeto MVC, `Views/Home/Index.cshtml`.

Está pronto quando: sua alteração aparece no navegador e você sabe dizer qual arquivo controla o título da aba.

5.2 Novo Controller e View

●○○

Crie `ProdutoController` com o método `Index()` e a view correspondente em `Views/Produto/Index.cshtml`. Acesse `/Produto`.

Treina: a convenção `Views/<Controller>/<Action>.cshtml`.

Está pronto quando: a URL `/Produto` abre sua página sem erro 404.

5.3 Passar dados para a View

●●○

No `ProdutoController`, crie uma lista de produtos em memória e envie para a view com `return View(lista)`. Na view, declare o `@model` e monte uma tabela HTML com `@foreach`.

```
@model List<LojaWeb.Models.Produto>

@foreach (var p in Model)
{
    <tr><td>@p.Nome</td><td>@p.Preco</td></tr>
}
```

Treina: Razor, `@model`, `@foreach`.

Está pronto quando: a tabela aparece preenchida com todos os produtos da lista.

5.4 Rota com parâmetro

●●○

Crie a action `Detalhes(int id)` que busca o produto na lista e mostra os dados dele em uma view própria. Se o id não existir, devolva `NotFound()`. Teste com `/Produto/Detalhes/2`.

Treina: parâmetro de rota, `NotFound()`.

Está pronto quando: `/Produto/Detalhes/999` devolve 404 e `/Produto/Detalhes/1` mostra o produto.

5.5 Navegação entre páginas ●●○

Na tabela do 5.3, adicione em cada linha um link "Detalhes" apontando para a action do 5.4. Adicione também "Produtos" no menu do `_Layout.cshtml`.

```
<a asp-controller="Produto" asp-action="Detalhes" asp-route-id="@p.Id">Detalhes</a>
```

Treina: tag helpers de link, `_Layout.cshtml`.

Está pronto quando: dá para navegar da lista para o detalhe e voltar usando só o mouse.

5.6 CSS próprio ●●○

Adicione estilos em `wwwroot/css/site.css` para deixar a tabela de produtos apresentável (linhas zebradas, cabeçalho destacado, espaçamento). Depois, **de propósito**, mova o arquivo CSS para fora do `wwwroot` e veja o que acontece.

Treina: por que o `wwwroot` existe.

Está pronto quando: você consegue explicar por que o CSS parou de funcionar fora do `wwwroot`.

5.7 Formulário ●●●

Crie a action `Criar()` (GET, mostra o formulário) e `Criar(Produto produto)` (POST, recebe os dados e adiciona na lista). Depois de salvar, redirecione para a listagem com `RedirectToAction("Index")`.

Treina: `[HttpGet]` / `[HttpPost]`, model binding, `RedirectToAction`.

Está pronto quando: preencher o formulário e enviar faz o produto aparecer na tabela.

5.8 Validação ●●●

Adicione anotações na classe `Produto` (`[Required]`, `[Range(0.01, 99999)]`, `[StringLength(80)]`). No POST, verifique `ModelState.IsValid` e, se inválido, devolva o formulário com as mensagens de erro.

Treina: Data Annotations, `ModelState`, `asp-validation-for`.

Está pronto quando: enviar o formulário vazio mostra as mensagens de erro e **não** adiciona nada na lista.

Nível 6 — Banco de dados com EF Core + SQLite

Trocar a lista em memória por um banco de verdade. Continue no projeto `LojaWeb`.

6.1 Instalar e configurar

●○○

Instale os pacotes, crie `Data/LojaDbContext.cs` com um `DbSet<Produto>`, adicione a connection string no `appsettings.json` e registre o contexto no `Program.cs`.

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite -v 8.0.10
dotnet add package Microsoft.EntityFrameworkCore.Design -v 8.0.10
dotnet tool install --global dotnet-ef --version 8.*
```

Treina: pacotes NuGet, `DbContext`, `AddDbContext`.

Está pronto quando: `dotnet build` passa sem erro.

6.2 Primeira migration

●●○

Gere a migration e aplique. Depois, abra a pasta `Migrations/` e leia o método `Up()` — identifique o comando que cria a tabela e quais colunas ele declara.

```
dotnet ef migrations add CriaTabelaProduto
dotnet ef database update
```

Treina: migrations, entender o código gerado.

Está pronto quando: o arquivo `.db` aparece na pasta e você consegue apontar no `Up()` qual linha cria cada coluna.

6.3 CRUD no banco

●●●

Refaça o `ProdutoController` para usar o banco em vez da lista em memória: listar, detalhar, criar, editar e excluir. Receba o `LojaDbContext` pelo construtor.

```
private readonly LojaDbContext _context;
public ProdutoController(LojaDbContext context) => _context = context;
```

Treina: injeção de dependência automática, `Add`, `Update`, `Remove`, `SaveChanges`.

Está pronto quando: você cadastra um produto, **para o servidor, roda de novo** e o produto continua lá.

6.4 Relacionamento 1:N ●●●

Crie a entidade `Categoria` (Id, Nome) e ligue: uma categoria tem vários produtos; um produto pertence a uma categoria. Gere a migration. Na listagem, mostre o nome da categoria de cada produto usando `Include`.

```
var produtos = _context.Produtos.Include(p => p.Categoria).ToList();
```

Treina: chave estrangeira, propriedade de navegação, `Include`.

Está pronto quando: a coluna "Categoria" aparece preenchida — e você testa remover o `Include` para ver o que acontece.

6.5 Seed ●●○

Use `HasData` no `OnModelCreating` para cadastrar automaticamente 4 categorias. Gere uma nova migration e aplique.

Treina: `HasData`, migration de dados.

Está pronto quando: você apaga o `.db`, roda `database update` e as 4 categorias já estão lá.

6.6 Migration de alteração ●●○

Adicione a propriedade `DateTime DataCadastro` na `Produto`. Gere e aplique uma nova migration chamada `AdicionaDataCadastro`. Depois rode `dotnet ef migrations list` e veja as três migrations.

Treina: evolução do esquema sem perder dados.

Está pronto quando: os produtos que já existiam continuam no banco depois da migration.

6.7 Relacionamento N:N ●●●

Crie a entidade `Tag` (Id, Nome) e relacione com `Produto` em muitos-para-muitos: um produto pode ter várias tags e uma tag pode estar em vários produtos. Gere a migration e observe a tabela de junção criada.

```
public ICollection<Tag> Tags { get; set; } = new List<Tag>();
```

Treina: N:N, tabela de junção automática do EF Core 8.

Está pronto quando: você identifica no banco a terceira tabela (`ProdutoTag`) que o EF criou sozinho.

6.8 Consultas no banco ●●●

Crie uma action `Relatorio()` que mostre: total de produtos, valor total do estoque, produtos agrupados por categoria com a contagem, e os 5 mais caros. Tudo com LINQ direto no `DbContext`.

Treina: LINQ traduzido para SQL, `GroupBy` no banco, `Take`.

Está pronto quando: a página do relatório mostra as quatro informações corretas.

Nível 7 — API REST

O mesmo CRUD, mas devolvendo JSON em vez de HTML. Crie `Controllers/ProdutosApiController.cs`.

7.1 Primeiro endpoint ●○○

Crie um controller de API que herde de `ControllerBase`, com `[ApiController]` e `[Route("api/produtos")]`. Implemente o `GET` que devolve todos os produtos. Acesse `/api/produtos` no navegador.

Treina: `[ApiController]`, `[Route]`, `[HttpGet]`, `Ok()`.

Está pronto quando: o navegador mostra um JSON com a lista de produtos.

7.2 GET por id ●●○

Implemente `GET /api/produtos/{id}`. Se o produto não existir, devolva `NotFound()`.

Treina: parâmetro de rota na API, status 404.

Está pronto quando: `/api/produtos/999` devolve status 404 (veja na aba Network do F12).

7.3 POST ●●○

Implemente `POST /api/produtos` que recebe um produto em JSON, salva e devolve `CreatedAtAction` com status 201. Teste usando um arquivo `.http` no VS Code (extensão REST Client) ou o Swagger.

```
POST http://localhost:5000/api/produtos
Content-Type: application/json

{ "nome": "Mouse", "preco": 89.90, "quantidade": 5 }
```

Treina: `[HttpPost]`, corpo da requisição, status 201.

Está pronto quando: a resposta é 201 e o produto aparece no `GET` seguinte.

7.4 PUT e DELETE ●●○

Implemente a atualização (`PUT /api/produtos/{id}`, devolvendo o objeto atualizado) e a exclusão (`DELETE /api/produtos/{id}`, devolvendo `NoContent()`). Ambos devem devolver 404 se o id não existir.

Treina: os quatro verbos do CRUD, status 204.

Está pronto quando: os quatro verbos funcionam e cada um devolve o status correto.

7.5 Validação na API ●●○

Rejeite requisições inválidas devolvendo `BadRequest` com uma mensagem clara: nome vazio, preço negativo, quantidade negativa.

Treina: status 400, validação no servidor.

Está pronto quando: enviar `{"nome": "", "preco": -5}` devolve 400 com a explicação do problema.

7.6 DTO

●●●

Crie `ProdutoDto` (só Id, Nome, Preco e NomeCategoria) e faça a API devolver o DTO em vez da entidade. Crie também `ProdutoCriarDto` para a entrada (sem Id).

Treina: DTO — separar o que o banco guarda do que a API expõe. Evita vazar dados e quebrar o cliente quando a entidade muda.

Está pronto quando: o JSON de resposta **não** tem propriedades de navegação nem campos internos.

7.7 Async

●●○

Converta todos os endpoints para assíncronos: `async Task<IActionResult>`, usando `ToListAsync()`, `FindAsync()` e `SaveChangesAsync()`.

Treina: `async` / `await`, versões assíncronas do EF Core.

Está pronto quando: nenhum endpoint tem chamada síncrona ao banco e tudo continua funcionando igual.

7.8 API de categorias

●●●

Repita todo o CRUD para `Categoria`, em `/api/categorias`. Adicione um endpoint extra: `GET /api/categorias/{id}/produtos`, que lista os produtos daquela categoria.

Treina: repetição do padrão (é assim que se fixa), rota aninhada.

Está pronto quando: você consegue montar o CRUD inteiro sem consultar o do produto.

Nível 8 — Frontend consumindo a API

JavaScript com `fetch` na pasta `wwwroot/js/`. Crie uma página que funcione **sem recarregar**.

8.1 Listar com `fetch` ●○○

Crie a view `Views/Produto/App.cshtml` com uma `<table>` vazia e o arquivo `wwwroot/js/produtos.js`. No JS, busque `/api/produtos` e preencha a tabela.

```
async function carregar() {
    const resp = await fetch("/api/produtos");
    const produtos = await resp.json();
    // ... monta as linhas
}
carregar();
```

Treina: `fetch`, `async/await`, manipulação do DOM.

Está pronto quando: a tabela aparece preenchida ao abrir a página, sem nenhum dado no HTML.

8.2 Criar com POST ●●○

Adicione campos de entrada e um botão "Adicionar". Ao clicar, envie um `POST` para a API e, se der certo, recarregue a lista e limpe os campos.

```
await fetch("/api/produtos", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ nome, preco, quantidade })
});
```

Treina: `fetch` com método, headers e body.

Está pronto quando: o produto aparece na tabela sem a página recarregar. **Teste esquecer o header `Content-Type`** e veja o erro que dá.

8.3 Excluir ●●○

Adicione um botão "Excluir" em cada linha. Peça confirmação com `confirm()` antes de enviar o `DELETE`.

Treina: `DELETE` via fetch, passar o id do item clicado.

Está pronto quando: excluir remove da tela e o item não volta ao recarregar a página.

8.4 Editar ●●●

Adicione "Editar" em cada linha: ao clicar, preencha o formulário com os dados atuais e mude o botão para "Salvar alteração", que envia um `PUT`. Depois de salvar, volte ao modo de criação.

Treina: `PUT`, controlar estado da tela (criando vs editando).

Está pronto quando: dá para criar, editar e criar de novo sem recarregar nem se perder no estado.

8.5 Tratamento de erro ●●○

Em toda chamada, verifique `resp.ok` antes de usar os dados. Se der erro, mostre a mensagem vinda da API em uma área de aviso na tela (não use `alert`). Adicione também um texto "Carregando..." enquanto a requisição está em andamento.

Treina: `response.ok`, feedback ao usuário — o que separa um sistema amador de um decente.

Está pronto quando: tentar cadastrar um produto inválido mostra o erro da API na tela, sem quebrar nada.

8.6 Busca e filtro ●●○

Adicione um campo de busca que filtre a tabela conforme o usuário digita. Faça duas versões: filtrando no navegador (sobre os dados já carregados) e filtrando no servidor (`/api/produtos?busca=xxx`).

Treina: evento `input`, query string, filtro no `Where` do backend.

Está pronto quando: as duas versões funcionam e você consegue explicar quando usar cada uma.

8.7 Deixar apresentável ●●●

Estilize a página: tabela limpa, formulário organizado, botões com cores distintas (adicionar/editar/excluir), mensagens de erro em vermelho e sucesso em verde. Garanta que funciona em tela de celular.

Treina: CSS, responsividade — a entrega de 23/10 do projeto real.

Está pronto quando: a página fica utilizável ao reduzir a janela para largura de celular (F12 → modo dispositivo).

Nível 9 — Um pouco além

Conteúdo que não vai cair na prova, mas que aparece no projeto e em qualquer sistema real.

9.1 Camada de serviço

Crie `Services/IProdutoService.cs` e `Services/ProdutoService.cs`. Mova para lá **toda** a lógica que hoje está no controller (validações, cálculos, acesso ao contexto). Registre no `Program.cs` e injete a interface no controller.

```
builder.Services.AddScoped<IProdutoService, ProdutoService>();
```

Treina: separação de responsabilidades, DI do ASP.NET Core. Conecta com o exercício 3.8.

Está pronto quando: o controller não menciona o `DbContext` em nenhuma linha.

9.2 Consumir uma API externa

Use `HttpClient` para consumir a **ViaCEP** (`https://viacep.com.br/ws/01001000/json/`). Crie um endpoint `GET /api/cep/{cep}` que busca lá e devolve o endereço. Trate CEP inválido.

```
builder.Services.AddHttpClient();   // no Program.cs
```

Treina: `HttpClient`, desserializar JSON externo, `async`. É exatamente o que o importador da PokéAPI faz.

Está pronto quando: `/api/cep/01001000` devolve "Praça da Sé" e um CEP inexistente devolve erro tratado.

9.3 Importador com anti-duplicação

Crie um endpoint `POST /api/importar` que busca dados de uma API externa e **salva no seu banco**. Regra: rodar duas vezes não pode duplicar registros — verifique antes de inserir e devolva quantos foram criados e quantos já existiam.

Treina: idempotência, `Any()` antes de inserir, índice único.

Está pronto quando: a segunda execução devolve "0 criados, N já existentes" e o total no banco não muda.

9.4 Paginação e ordenação

Altere `GET /api/produtos` para aceitar `?pagina=1&tamanho=10&ordenarPor=preco`. Devolva também o total de registros e o total de páginas. No frontend, adicione botões "anterior" e "próxima".

```
.Skip((pagina - 1) * tamanho).Take(tamanho)
```

Treina: `Skip` / `Take`, query string com vários parâmetros, resposta com metadados.

Está pronto quando: com 25 produtos e tamanho 10, a página 3 devolve 5 itens e informa "3 de 3".

9.5 Teste automatizado

Desafio

Crie um projeto de testes (`dotnet new xunit -n LojaWeb.Tests`) e escreva testes para o `ProdutoService` : cadastrar com preço negativo deve falhar; cadastrar válido deve funcionar; buscar id inexistente deve devolver nulo.

Treina: xUnit, `[Fact]` , `Assert` , banco em memória para teste.

Está pronto quando: `dotnet test` roda e mostra os três testes passando.

Projeto integrador — Sistema de Biblioteca

Junta tudo. Faça do zero, em um projeto novo, sem copiar dos exercícios anteriores.

Construa um sistema web para uma biblioteca gerenciar seu acervo e os empréstimos.

Modelo de dados exigido

Entidade	Campos e relacionamentos
`Autor`	Id, Nome, Nacionalidade. **1:N** com Livro.
`Livro`	Id, Titulo, Ano, Isbn (único), QuantidadeExemplares, AutorId. **N:N** com Categoria.
`Categoria`	Id, Nome (único). **N:N** com Livro.
`Membro`	Id, Nome, Email (único), DataCadastro.
`Emprestimo`	Id, LivroId, MembroId, DataEmprestimo, DataPrevistaDevolucao, DataDevolucao (nulo enquanto não devolveu), Status (enum).

Requisitos funcionais

1. CRUD completo (API + tela) para **Autor**, **Livro**, **Categoria** e **Membro**.
2. **Registrar empréstimo:** só pode se houver exemplar disponível. A data prevista de devolução é 14 dias após o empréstimo.
3. **Registrar devolução:** preenche a data de devolução e libera o exemplar.
4. **Regra de bloqueio:** um membro com empréstimo atrasado não pode pegar outro livro.
5. **Limite:** no máximo 3 empréstimos simultâneos por membro.
6. **Tela de empréstimos:** lista com filtro por situação (em dia / atrasado / devolvido), destacando os atrasados em vermelho.
7. **Busca de livros** por título ou autor, com paginação.
8. **Relatório:** livros mais emprestados, membros com mais empréstimos e total de livros por categoria.

Requisitos técnicos

- ASP.NET Core MVC (.NET 8) + EF Core + SQLite, com migrations versionadas.
- Camada de serviço com interface: **nenhuma regra de negócio dentro do controller**.
- API REST com DTOs, todos os endpoints `async` e status HTTP corretos.
- Frontend consumindo a API com `fetch`, com tratamento de erro e estado de carregamento.
- Seed inicial com pelo menos 5 categorias, 5 autores e 10 livros.
- `README.md` explicando como rodar.
- Repositório no GitHub com commits ao longo do desenvolvimento (não um commit único no final).

> **Como avaliar se ficou bom:** peça para um colega clonar o repositório e rodar seguindo só o README. Se ele conseguir sem te perguntar nada, o projeto está pronto.

Extensões opcionais

- Reserva de livro quando não há exemplar disponível, com fila de espera.
- Cálculo de multa por atraso (valor por dia).

- Histórico de empréstimos por membro, com exportação em CSV.
- Autenticação simples separando bibliotecário de membro.

Folha de acompanhamento

Imprima ou copie para marcar o progresso. Sugestão: preencha a data de conclusão de cada nível.

Nível	Conteúdo	Exercícios	Concluído em	Dificuldade sentida
0	Preparação do ambiente e Git	0.1 – 0.3		
1	Fundamentos de C#	1.1 – 1.9		
2	POO parte 1	2.1 – 2.8		
3	POO parte 2 (herança, interfaces)	3.1 – 3.8		
4	Coleções e LINQ	4.1 – 4.8		
5	MVC: Controllers e Views	5.1 – 5.8		
6	EF Core + SQLite	6.1 – 6.8		
7	API REST	7.1 – 7.8		
8	Frontend com fetch	8.1 – 8.7		
9	Serviços, API externa, paginação	9.1 – 9.5		
—	Projeto integrador: Biblioteca	—		

Onde consultar quando travar

Assunto	Fonte
Sintaxe de C#	Documentação oficial da Microsoft (learn.microsoft.com/dotnet/csharp) — tem exemplos executáveis.
EF Core e migrations	learn.microsoft.com/ef/core
Estrutura MVC e API	O material da disciplina: "Guia do Projeto — MVC em C#".
Erro específico	Copie a **mensagem completa** do erro e pesquise. A maior parte já aconteceu com outra pessoa.
Como fizemos no projeto	O Manual Técnico do PokéDraft — tem exemplo real de entidade, DbContext, controller de API e JavaScript.

> **Conselho final:** não pule os níveis 3 e 4. Herança, interfaces e LINQ são o que separa "consigo fazer funcionar" de "consigo fazer direito" — e são exatamente os pontos que aparecem na hora de organizar o projeto em camadas.